Microsoft

Hostbased certificate selection

Serverside SNI in PostgreSQL 19

Daniel Gustafsson

PGConf.dev 2026 - Vancouver, Canada



Previously

vmware™ Pivotal

https://archive.org/details/office-43



User since 5/3 3PM, 2005 (#blamebruce)

Contributing since ~ 2013

Major Contributor and Committer (postgres/pgweb)

https://archive.org/details/office-43



https://archive.org/details/office-43

User since 5/3 3PM, 2005 (#blamebruce)

Contributing since ~ 2013

Major Contributor and Committer (postgres/pgweb)

curl://

REVERT

ADVISORY

WARNING

SNI?

SERVER NAME INDICATION

Extension to the TLS protocol. Supports TLS 1.0+, standardised in RFC 3546 (2003)

Client specifies which hostname to connect to in the TLS handshake

Server can present different certificate depending on which host is requested for the same IP/port

Client



Server

192.168.1.1

https://www.foo.com



foo.key

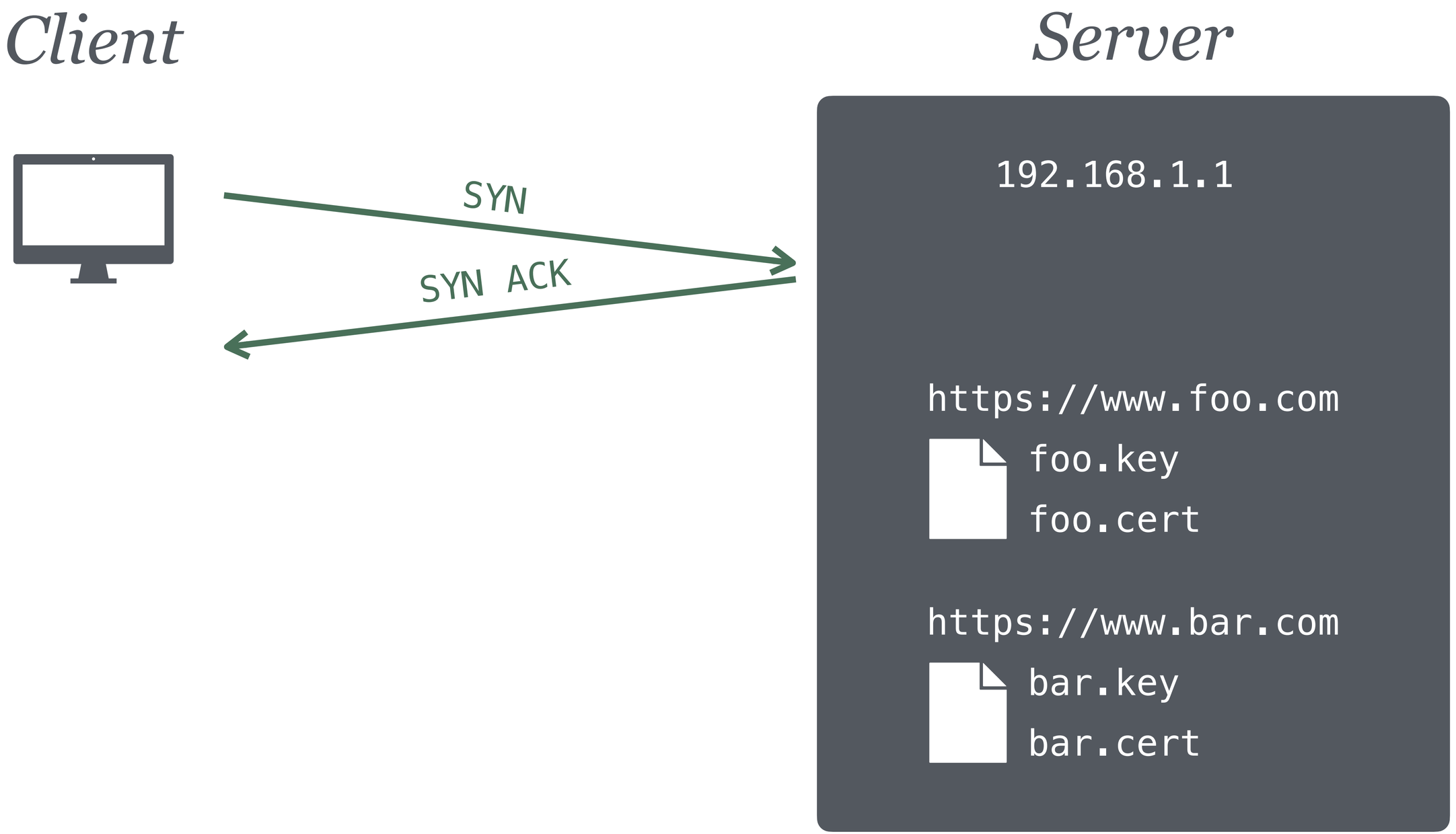
foo.cert

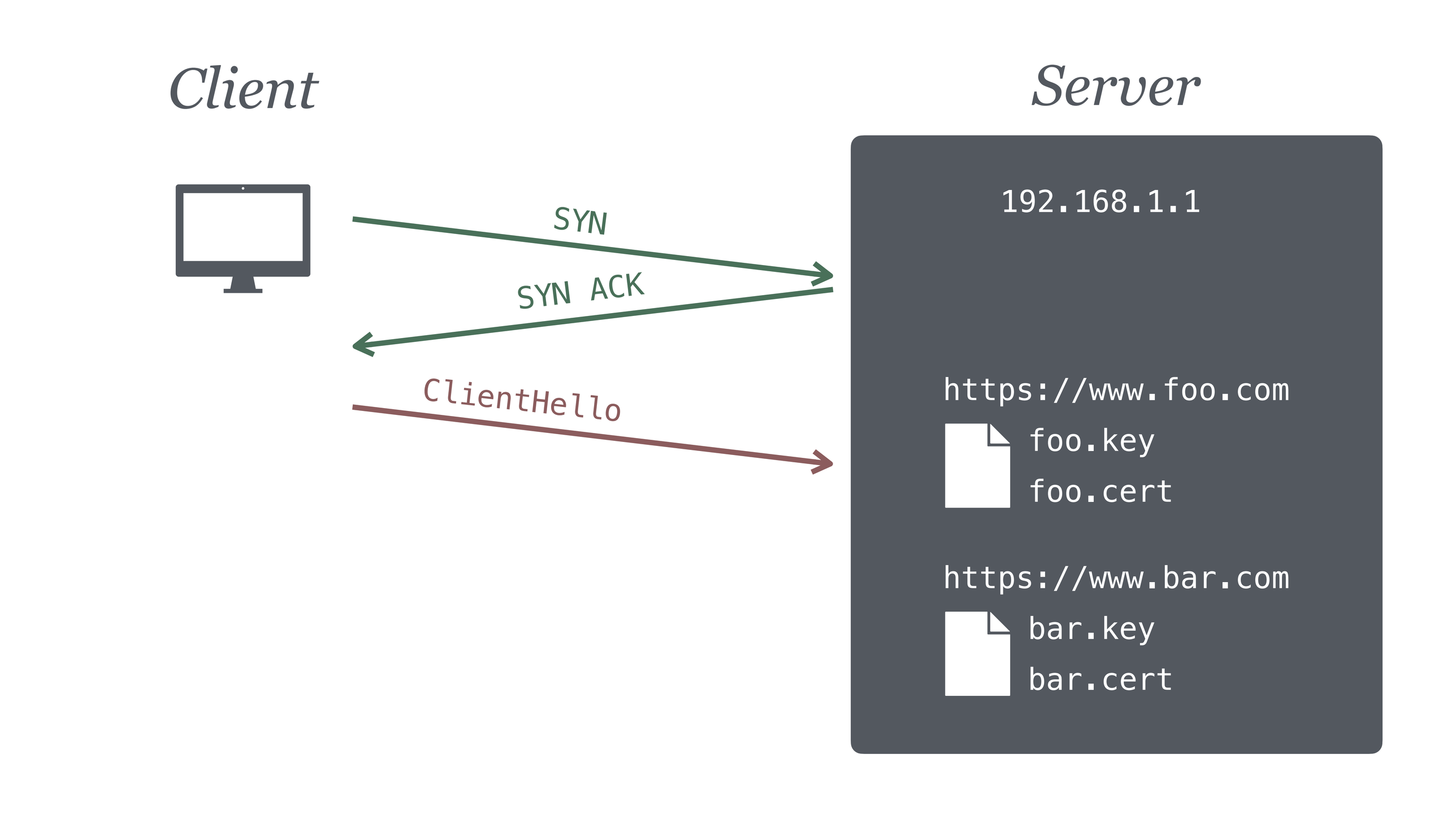
https://www.bar.com

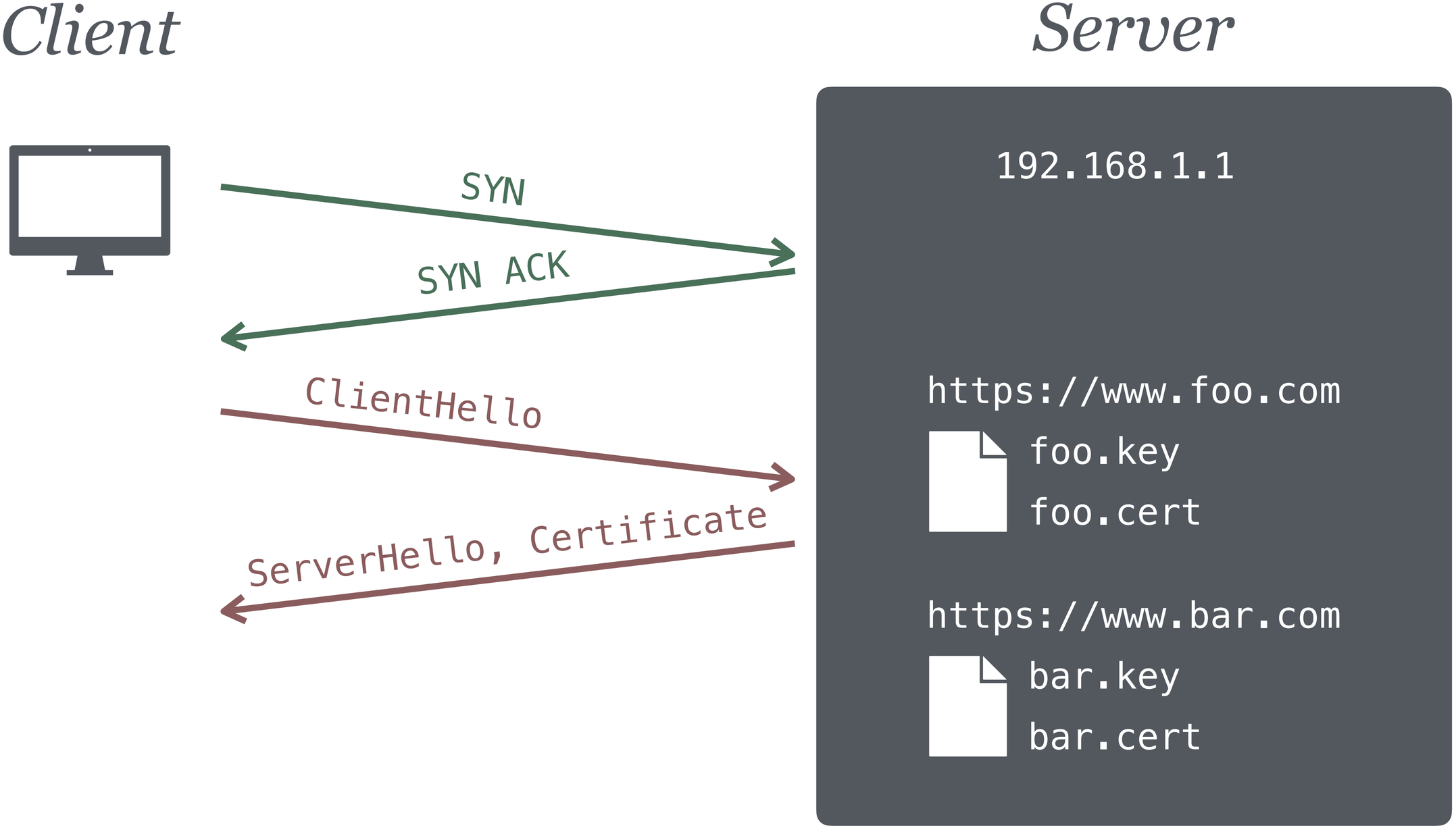


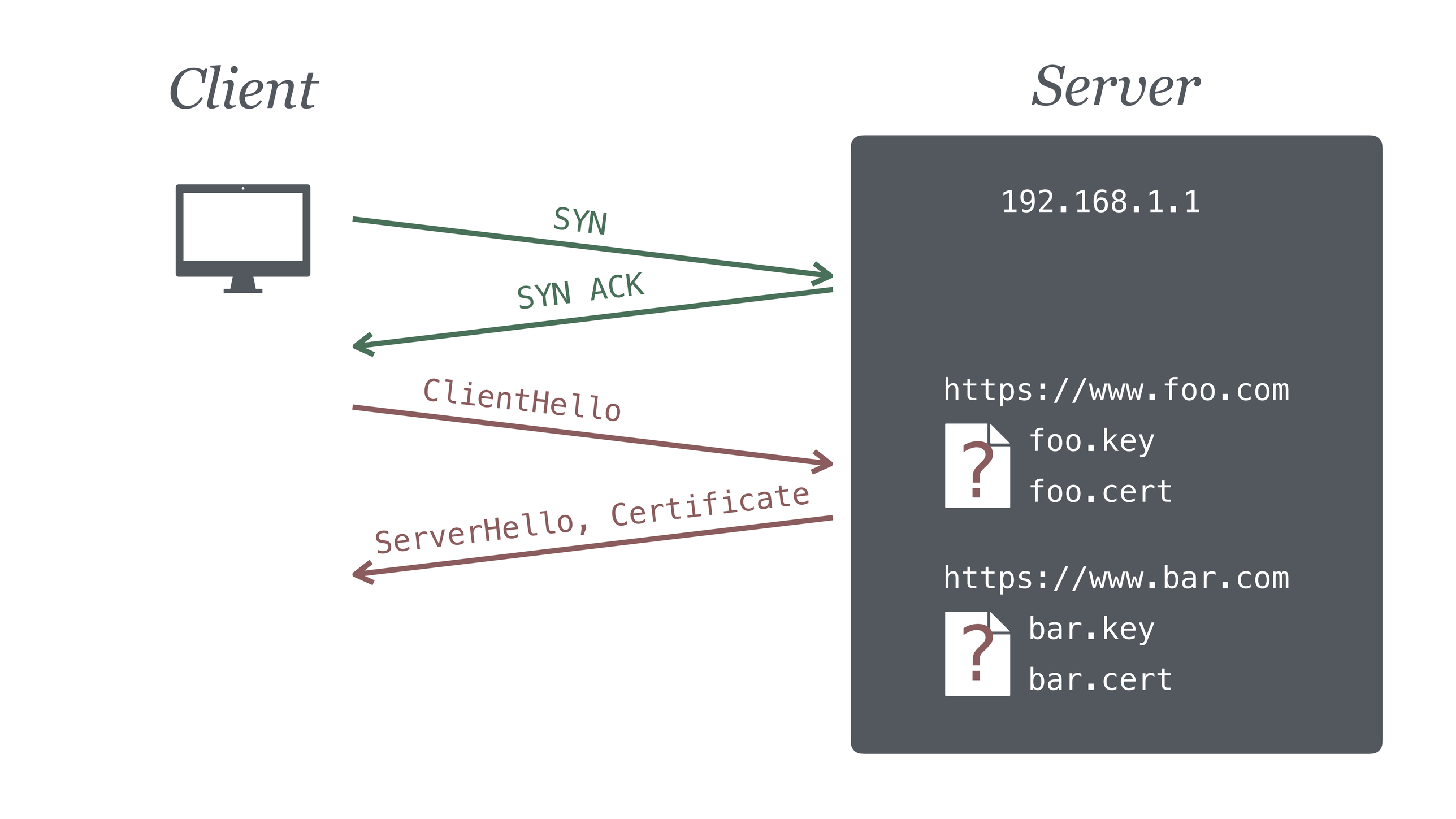
bar.key

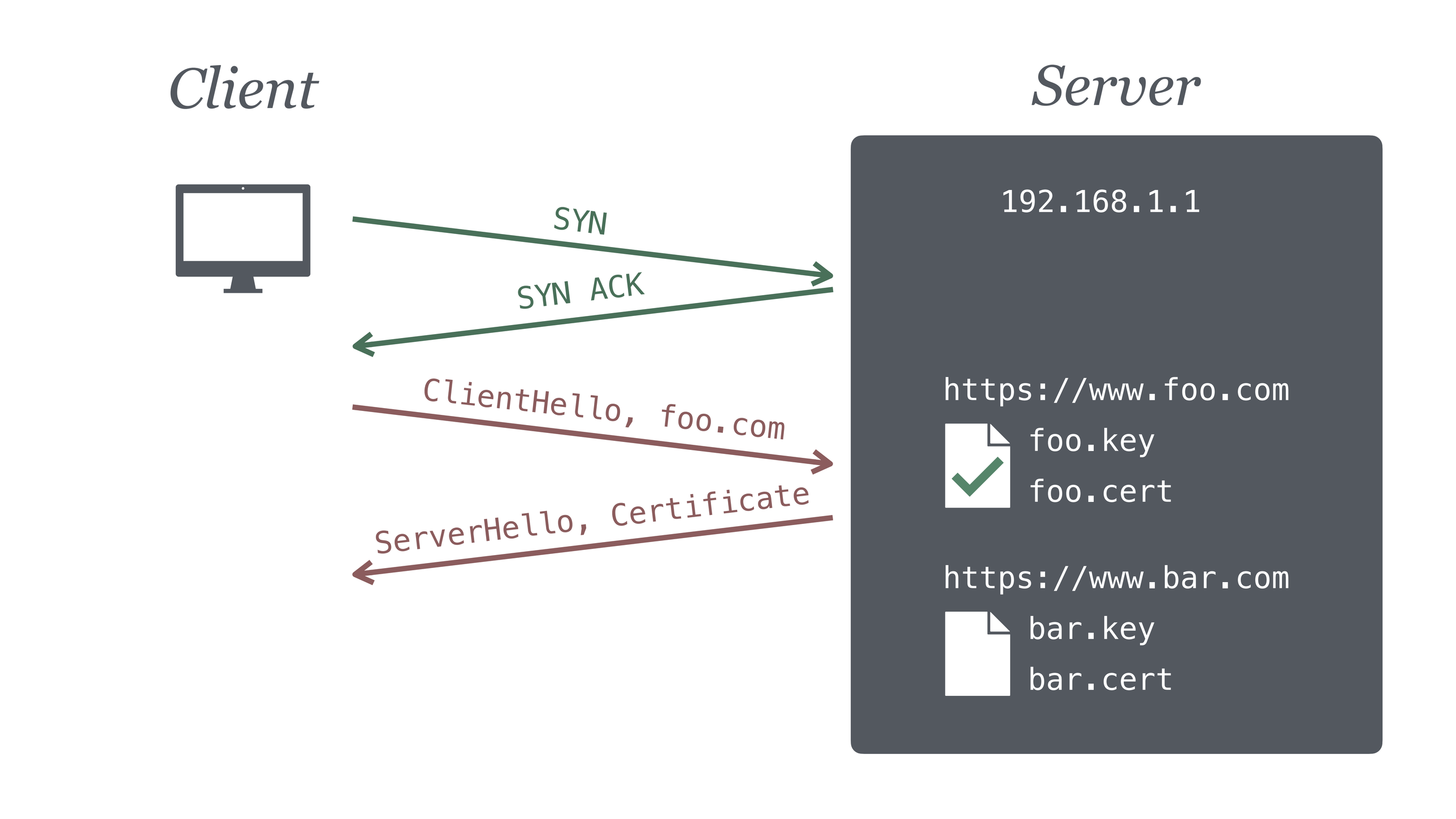
bar.cert


Client
Server
SYN
SYN ACK
192.168.1.1
https://www.foo.com
foo.key
foo.cert
https://www.bar.com
bar.key
bar.cert


Client
Server
192.168.1.1
SYN
SYN ACK
ClientHello
https://www.foo.com
foo.key
foo.cert
https://www.bar.com
bar.key
bar.cert


Client
Server
192.168.1.1
SYN
SYN ACK
ClientHello
ServerHello, Certificate
https://www.foo.com
foo.key
foo.cert
https://www.bar.com
bar.key
bar.cert


Client
Server
192.168.1.1
SYN
SYN ACK
ClientHello
ServerHello, Certificate
https://www.foo.com
foo.key
foo.cert
https://www.bar.com
bar.key
bar.cert


Client
Server
192.168.1.1
SYN
SYN ACK
ClientHello, foo.com
ServerHello, Certificate
https://www.foo.com
foo.key
foo.cert
https://www.bar.com
bar.key
bar.cert

Client

Server

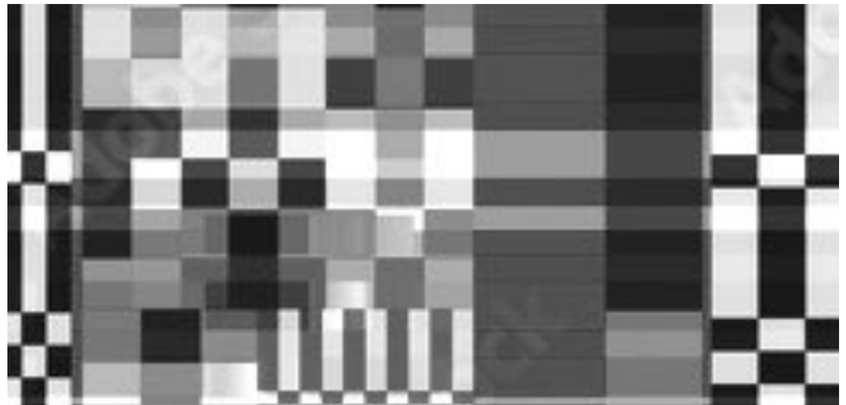
192.168.1.1

SYN

SYN ACK

ClientHello, foo.com

ServerHello, Certificate



https://www.foo.com

foo.key

foo.cert

https://www.bar.com

bar.key

bar.cert

Correct certificate presented for each connection

Multiple CA's

Multitenancy to the degree that the app can provide host-based isolation/routing

Client

Server

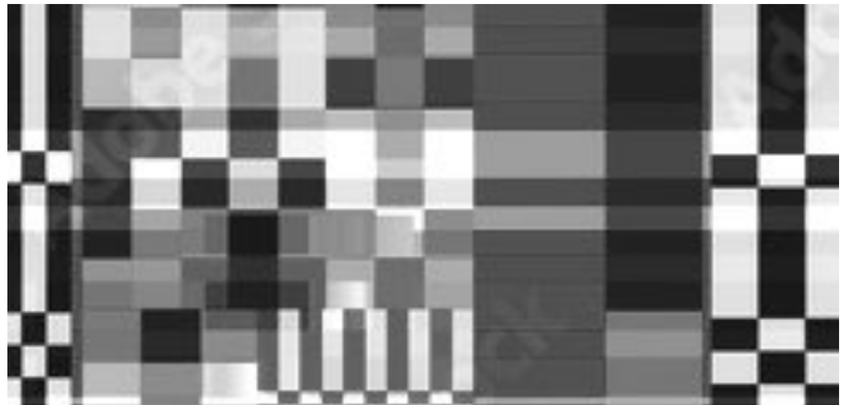
192.168.1.1

SYN

SYN ACK

ClientHello, foo.com

ServerHello, Certificate



https://www.foo.com

foo.key

foo.cert

https://www.bar.com

bar.key

bar.cert

Client

Server

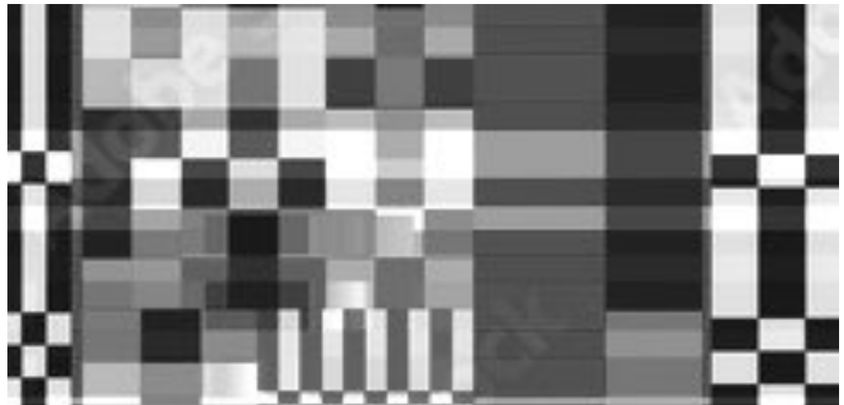


SYN

SYN ACK

ClientHello, foo.com

ServerHello, Certificate



192.168.1.1

https://www.foo.com



foo.key

foo.cert

https://www.bar.com



bar.key

bar.cert

SNI payload isn't encrypted

Hostname visible to sniffing/eavesdropping, subject to monitoring, blocking, shaping etc..

Multiple approaches for securing SNI

Encrypted Client Hello, RFC 9849 (March 2026)

PoSTGRES?

commit 5c55dc8b47338e72a4e598c155d2048d756fd10e
Author: Peter Eisentraut <peter@eisentraut.org>
Date: **Wed Apr 7 15:11:41 2021 +0200**

libpq: Set Server Name Indication (SNI) for SSL connections

By default, have libpq set the TLS extension "Server Name Indication" (SNI).

This allows an SNI-aware SSL proxy to route connections. (This
requires a proxy that is aware of the PostgreSQL protocol, not just
any SSL proxy.)

In the future, this could also allow the server to use different SSL
certificates for different host specifications. (That would require
new server functionality. This would be the client-side functionality
for that.)

Since SNI makes the host name appear in cleartext in the network
traffic, this might be undesirable in some cases. Therefore, also add
a libpq connection option "sslsni" to turn it off.

Allows libpq clients to send SNI in the handshake

Added a new boolean connection param `sslsni`

When set to 1, the hostname will be sent as the SNI extension. When set to 0 the extension isn't used

Clientside functionality for external tools

```
commit 5c55dc8b47338e72a4e598c155d2048d756fd10e
Author: Peter Eisentraut <peter@eisentraut.org>
Date:   Wed Apr 7 15:11:41 2021 +0200

    libpq: Set Server Name Indication (SNI) for SSL connections

    By default, have libpq set the TLS extension "Server Name Indication" (SNI).

    This allows an SNI-aware SSL proxy to route connections.  (This
    requires a proxy that is aware of the PostgreSQL protocol, not just
    any SSL proxy.)

    In the future, this could also allow the server to use different SSL
    certificates for different host specifications.  (That would require
    new server functionality.  This would be the client-side functionality
    for that.)

    Since SNI makes the host name appear in cleartext in the network
    traffic, this might be undesirable in some cases.  Therefore, also add
    a libpq connection option "sslsni" to turn it off.
```

> *In the future, this could also allow the server to use different SSL certificates for different host specifications. (That would require new server functionality. This would be the client-side functionality for that.)*
>
> *- Peter Eisentraut*


A Few
Moments Later

commit 4f433025f666fa4a6209f0e847715767fb1c7ace
Author: Daniel Gustafsson <dgustafsson@postgresql.org>
Date: **Wed Mar 18 12:37:11 2026 +0100**

ssl: Serverside SNI support for libpq

Support for SNI was added to clientside libpq in 5c55dc8b4733 with the sslsni parameter, but there was no support for utilizing it serverside. This adds support for serverside SNI such that certificate/key handling is available per host. A new config file, $datadir/pg_hosts.conf, is used for configuring which certificate and key should be used for which hostname. In order to use SNI the ssl_sni GUC must be set to on, when it is off the ssl configuration works just like before. If ssl_sni is enabled and pg_hosts.conf is non-empty it will take precedence over the regular SSL GUCs, if it is empty or missing the regular GUCs will be used just as before this commit with no hostname specific handling. The TLS init hook is not compatible with ssl_sni since it operates on a single TLS configuration and SNI break that assumption. If the init hook and ssl_sni are both enabled, a WARNING will be issued.

Adds a new GUC `ssl_sni` *as well as a new configuration file* `$datadir/pg_hosts.conf`

When disabled, configuration is unchanged

When enabled, allows different certificate/keys per hostname

SNI payload visible in cleartext

CRLs are applied per cluster, not per hostname

Supports OpenSSL 1.1.1+

Supports all versions of the TLS protocol

Does not work when built with LibreSSL

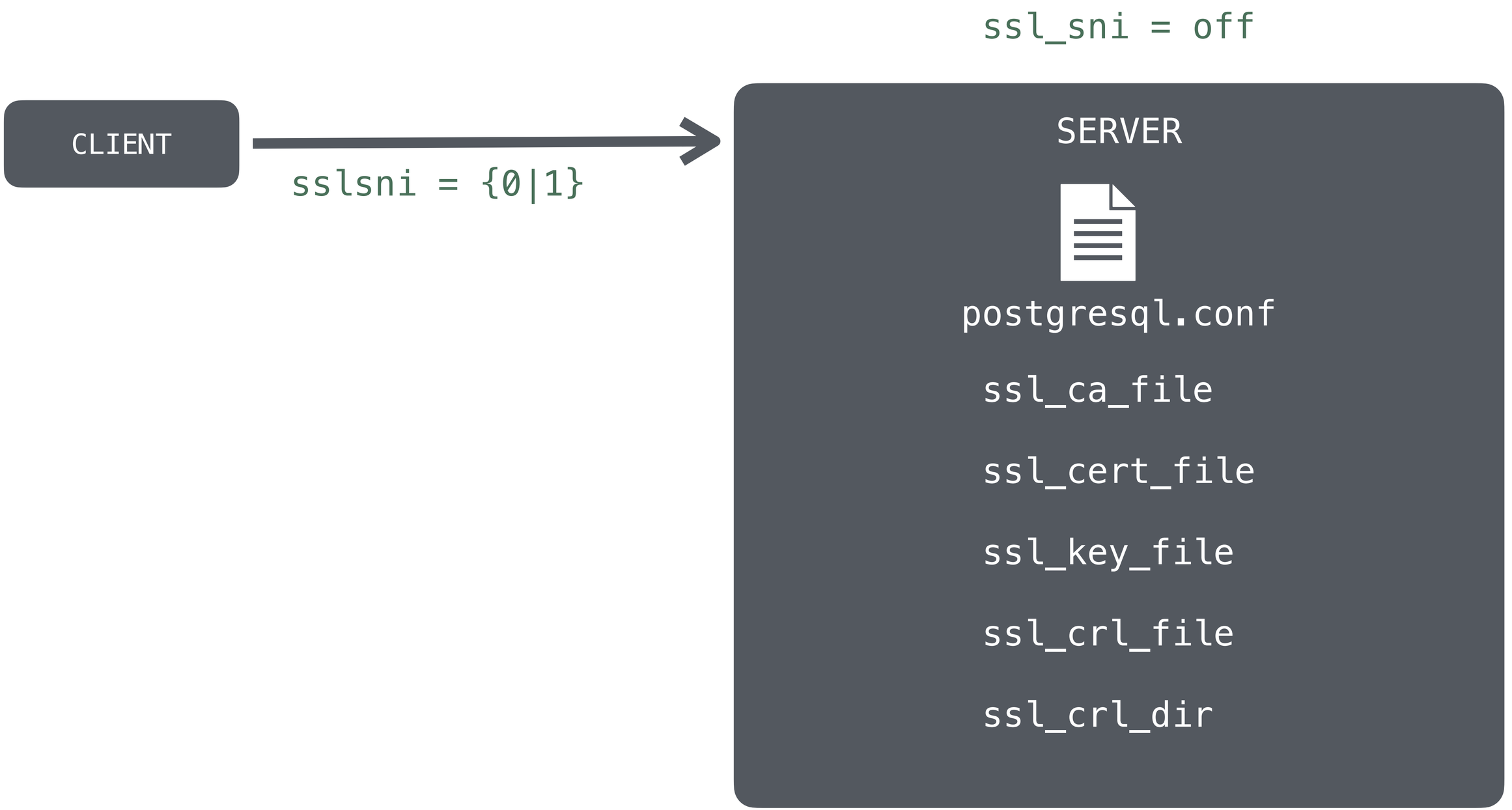
SNI payload visible in cleartext

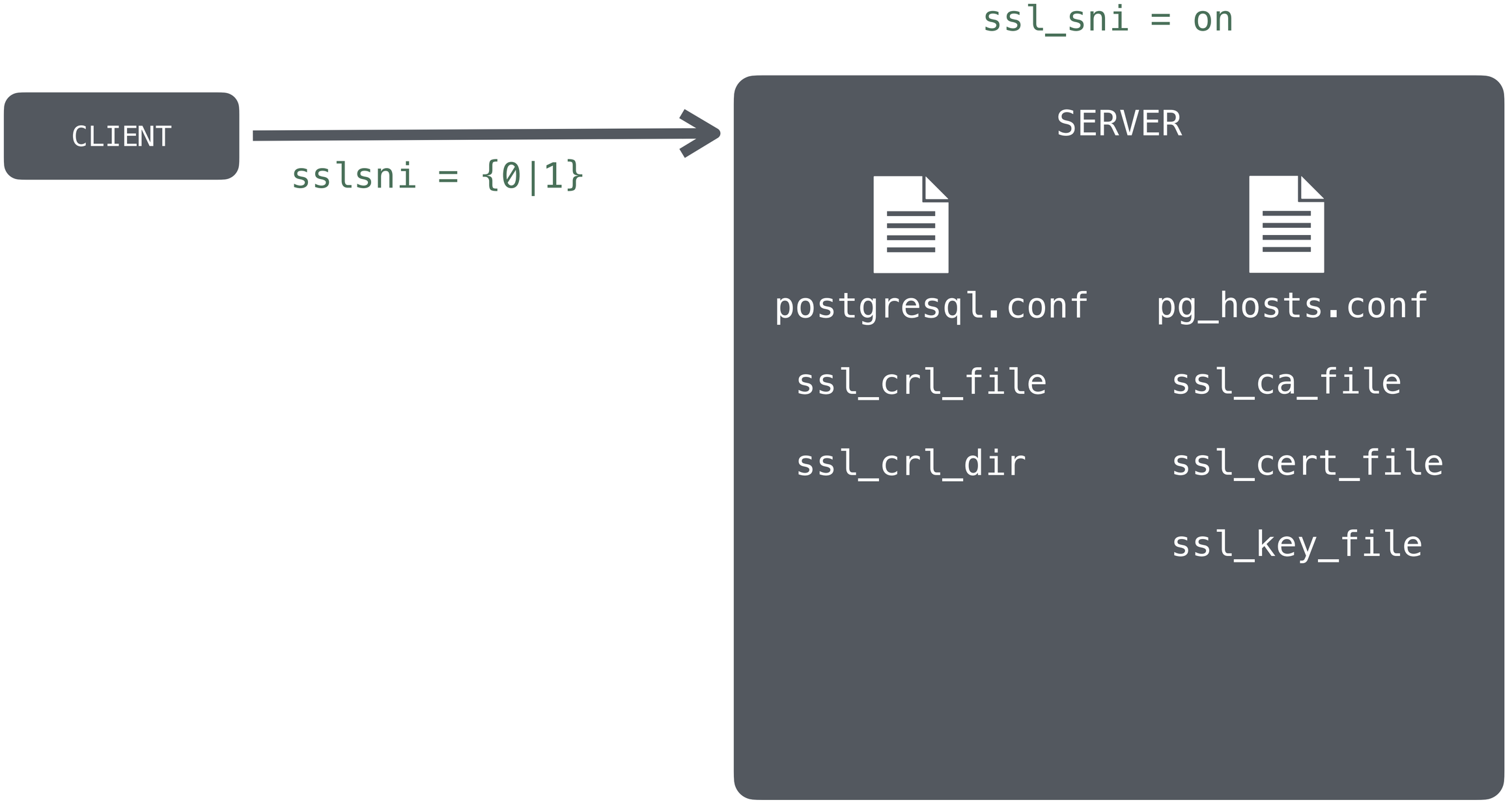
CRLs are applied per cluster, not per hostname

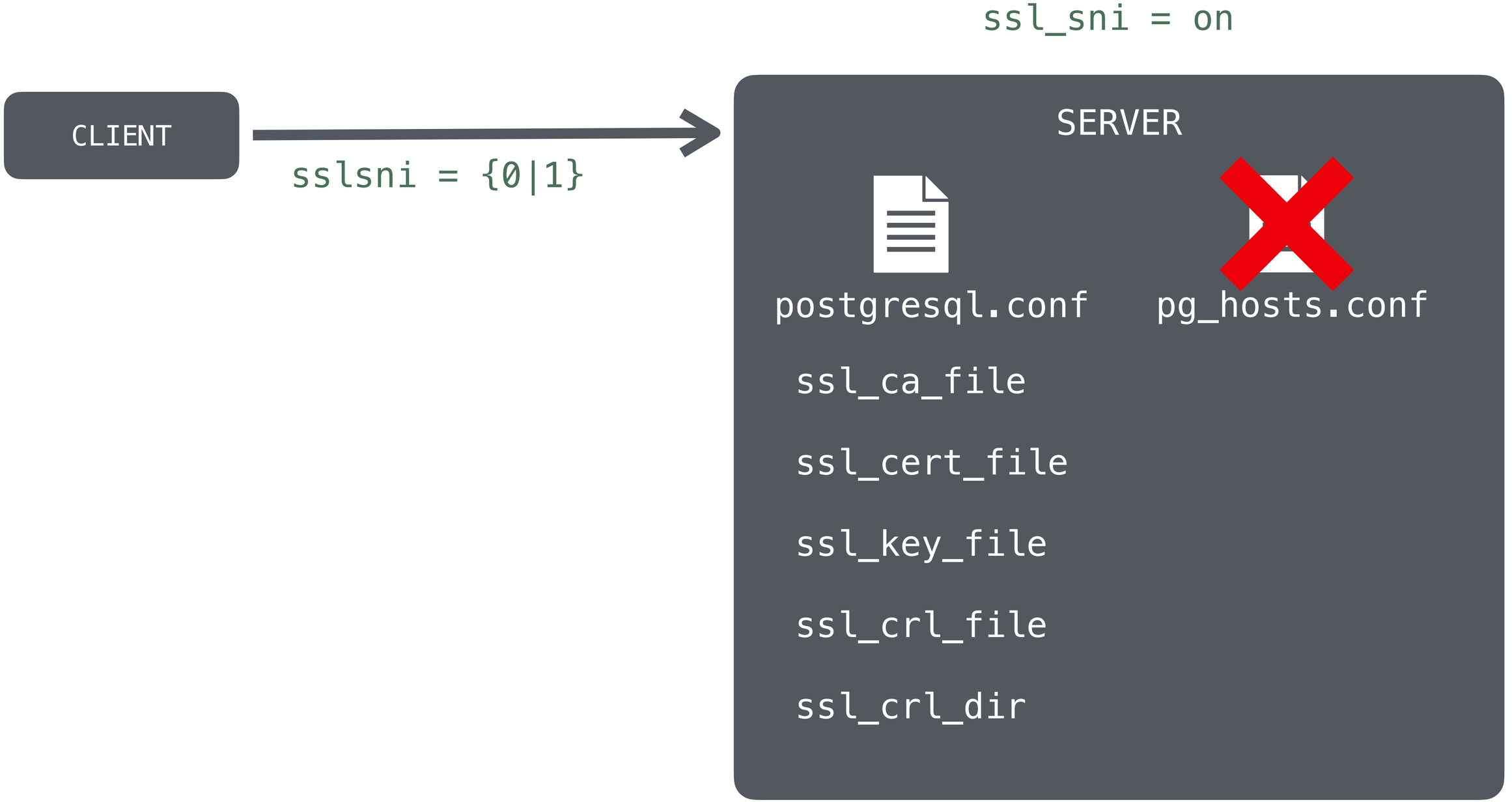
Supports OpenSSL 1.1.1+ *(please don't use 1.1.1)*

Supports all versions of the TLS protocol

Does not work when built with LibreSSL


ssl_sni = off
CLIENT
sslsni = {0|1}
SERVER
postgresql.conf
ssl_ca_file
ssl_cert_file
ssl_key_file
ssl_crl_file
ssl_crl_dir


ssl_sni = on
CLIENT
sslsni = {0|1}
SERVER
postgresql.conf
ssl_crl_file
ssl_crl_dir
pg_hosts.conf
ssl_ca_file
ssl_cert_file
ssl_key_file

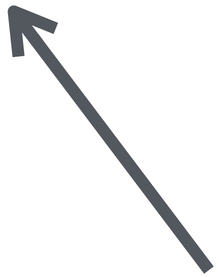

ssl_sni = on
CLIENT
sslsni = {0|1}
SERVER
postgresql.conf
pg_hosts.conf
ssl_ca_file
ssl_cert_file
ssl_key_file
ssl_crl_file
ssl_crl_dir

```
# PostgreSQL SNI Hostname mappings
# ================================

# HOSTNAME    SSL CERTIFICATE    SSL KEY    SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
```

```
# PostgreSQL SNI Hostname mappings
# ================================

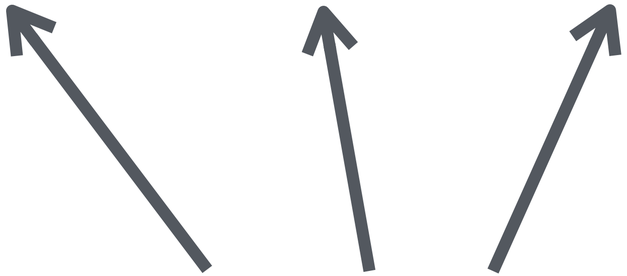
# HOSTNAME    SSL CERTIFICATE    SSL KEY    SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
```



Default host, `/no_sni/` or a list of hostnames

```
# PostgreSQL SNI Hostname mappings
# ================================

# HOSTNAME    SSL CERTIFICATE    SSL KEY    SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
```



Filenames of certificate and keys

```
# PostgreSQL SNI Hostname mappings
# ================================

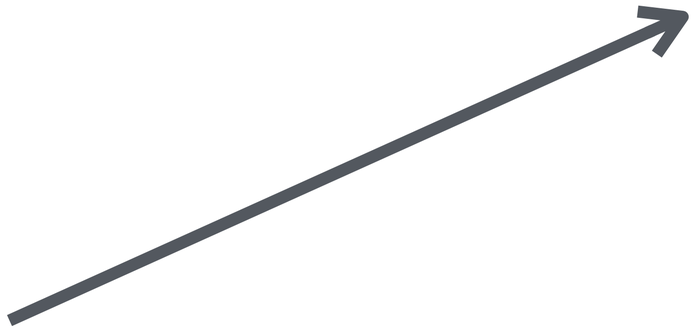
# HOSTNAME    SSL CERTIFICATE    SSL KEY    SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
```



SSL key passphrase command

```
# PostgreSQL SNI Hostname mappings
# ================================

# HOSTNAME    SSL CERTIFICATE    SSL KEY    SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
```



Can the SSL key passphrase command be reloaded?

Default host

Configured with a single asterisk: `*`

Matches all connection, regardless of SNI extension value or presence

Same functionality as pre-v19 servers

Configuration matching `sslsni=0` *connections*

Configured with keyword: `/no_sni/`

Keyword chosen to not collide with hostnames

Matches connections where the SNI extension is explicitly ***not present****

Per-host configuration *of keys/certificates*

Configured with a single hostname: `host`

Or using a list: `host_1,host_2,..,host_n`

Case insensitive hostname matching

No regular expressions

```
# PostgreSQL SNI Hostname mappings
# ================================

# HOSTNAME    SSL CERTIFICATE  SSL KEY       SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
*             default.cert     default.key   server.ca
foo.com       foo.cert         foo.key       wan.ca
bar.com       bar.cert         bar.key       wan.ca    "echo SeCrEt!"        Yes
alice         alice.cert       alice.key     private.ca
bob           bob.cert         bok.key       private.ca
/no_sni/      nosni.cert       nosni.key     server.ca
```

```
# PostgreSQL SNI Hostname mappings
# ================================

# HOSTNAME    SSL CERTIFICATE  SSL KEY       SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
*             default.cert     default.key   server.ca
foo.com       foo.cert         foo.key       wan.ca
bar.com       bar.cert         bar.key       wan.ca    "echo SeCrEt!"        Yes
alice         alice.cert       alice.key     private.ca
bob           bob.cert         bok.key       private.ca
/no_sni/      nosni.cert       nosni.key     server.ca
```



Order is irrelevant

Order of configuration has no impact on matching

Hostname takes precedence over default

Not configuring a default host or a `/no_sni/` *will create a "strict" mode where each connection must use SNI and must specify a defined hostname*

Does not offer multitenancy past the connection

Once connected, there is no separation of concern

Clients/users/departments connecting to different databases can be served by different CA's

LAN/WAN/DMZ specific CA's

InTERNALS

```
static struct hosts
{
    /*
     * List of HostsLine structures containing SSL configurations
     * for connections with hostnames defined in the SNI extension.
     */
    List        *sni;
    /* The SSL configuration to use for connections without SNI */
    HostsLine   *no_sni;
    /*
     * The default SSL configuration to use as a fallback in case no
     * hostname matches the supplied hostname in the SNI extension.
     */
    HostsLine   *default_host;
}           *SSL_hosts;
```

```
typedef struct HostsLine
{
    int          linenumber;
    char        *sourcefile;
    char        *rawline;
    /* Required fields */
    List        *hostnames;
    char        *ssl_key;
    char        *ssl_cert;
    /* Optional fields */
    char        *ssl_ca;
    char        *ssl_passphrase_cmd;
    bool         ssl_passphrase_reload;
    /* Internal bookkeeping */
    void        *ssl_ctx;
} HostsLine;
```

```
/*
 * sni_clienthello_cb
 *
 * Callback for extracting the servername extension
 * from the TLS handshake during ClientHello.  There
 * is a callback in OpenSSL for the servername
 * specifically but OpenSSL themselves advice against
 * using it as it is more dependent on ordering for
 * execution.
 */
static int
sni_clienthello_cb(SSL *ssl, int *al, void *arg)
{
  ...
}
```

```
/*
 * ssl_update_ssl
 *
 * Replace certificate/key and CA in an SSL object
 * to match the, via the SNI extension, selected
 * host configuration for the connection.  The
 * SSL_CTX object to use should be passed in as ctx.
 * This function will update the SSL object in-place.
 */
static bool
ssl_update_ssl(SSL *ssl, HostsLine *host_config)
{
    ...
}
```

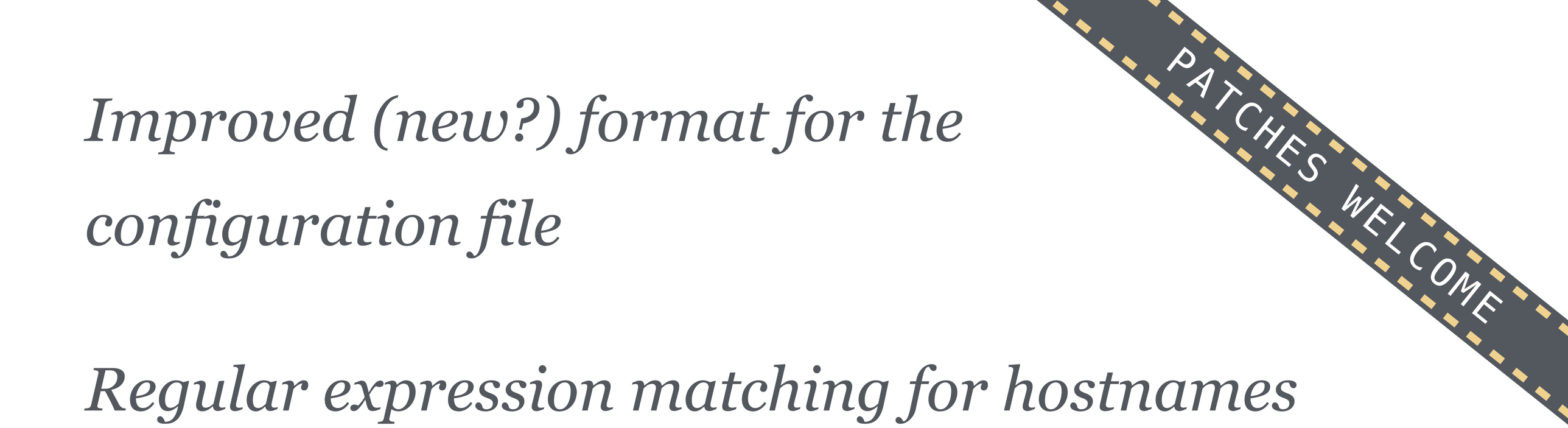
FuTURE WORK

Improved (new?) format for the configuration file

Regular expression matching for hostnames

Encrypted Client Hello (req. OpenSSL 4+)

LibreSSL support



TL;DR

*Clientside functionality in all supported versions, serverside coming in v19**

Without enabling, there is no change from the current way of configuring TLS, when enabled it can be used for separating security concerns

Patches welcome!

** assuming it's not getting reverted*

Microsoft

Thank you!

daniel@yesql.se